

# MERN/Next.js Full-Stack Project Ideas

## 1. Community Volunteer Coordination Platform

*Project:* A local volunteer management system where nonprofits or community groups can create and manage service events (clean-ups, food drives, etc.), and volunteers can browse, sign up, and track their hours. *Components:* React or Next.js frontend, Node/Express backend, MongoDB storage, and JWT-based authentication. *Features:* Event creation and scheduling (with date/time and location), volunteer sign-up and approval workflows, role-based user accounts (admins vs volunteers), and calendar integration (e.g. Google Calendar API). Notifications (email or push) alert organizers and volunteers of new events or updates. The system can use geolocation to match users with nearby events and include an analytics dashboard showing volunteer participation metrics <sup>1</sup>. This practical app showcases a full MERN stack with real-time (Socket.IO) or scheduled notifications, and is ideal for a GitHub portfolio as a community-impact project.

- **Frontend:** React/Next.js UI for browsing events and dashboards
- **Backend:** Node/Express API for events, users, and sign-ups
- **Database:** MongoDB collections for users, events, and volunteer records
- **Auth:** JWT or OAuth for secure login/roles
- **Key features:** Event CRUD; volunteer registration/approval; calendar/API integration; real-time or email notifications; analytics (e.g. hours served) <sup>1</sup>.

## 2. Neighborhood Safety & Alert System

*Project:* A location-based safety app for local communities. Residents can report incidents (crime, hazards, lost pets, etc.) and view a map of nearby safety alerts. *Components:* Next.js frontend (for SEO and performance), Node/Express backend, MongoDB, user authentication. *Features:* Geolocation or map interface showing reported incidents; user-submitted alerts with categories; community safety ratings and comments; admin dashboard for moderators. It could integrate public safety APIs (e.g. local police crime data) and send real-time alerts (push/SMS) to users in affected areas. An analytics dashboard could display trends (crime heatmap, types of incidents) over time. This MERN app mirrors a “Neighborhood Safety Rating” project <sup>2</sup> but extends to two-way reporting and emergency notifications, making it a useful tool for local users.

- **Frontend:** React/Next.js map and reporting UI
- **Backend:** Express APIs for incident reports and user data
- **Database:** MongoDB stores incidents, locations, user profiles
- **Auth:** Secure login for residents and moderators
- **Key features:** Geolocation (map view of incidents); CRUD for safety reports; sorting/rating neighborhoods; real-time alerts; analytics dashboard of safety trends <sup>2</sup>.

## 3. Team Task and Project Management Tool

*Project:* A collaborative task/Kanban board system for teams or personal productivity. Users can create boards, lists, and tasks (tickets), assign team members, set due dates, and track progress. *Components:* React or Next.js SPA, Express backend, MongoDB, JWT auth. *Features:* Drag-and-drop Kanban interface,

task metadata (labels, deadlines, attachments), user assignments, and project timelines. Integration with external tools (Google Calendar or Slack API for updates) can add polish. A dashboard can show productivity analytics (tasks completed per week, overdue tasks, etc.). Real-time updates via Socket.IO allow multiple users to see changes instantly (e.g. task moved to “Done” by another user). This full-stack app demonstrates advanced features like dynamic UI updates and is a great portfolio piece.

- **Frontend:** React/Next.js with drag-and-drop (e.g. react-beautiful-dnd)
- **Backend:** Node/Express for project and task APIs
- **Database:** MongoDB for projects, tasks, user profiles
- **Auth:** JWT or OAuth2 for team accounts
- **Key features:** Project Kanban boards; real-time task updates (Socket.IO); file attachments for tasks; calendar deadlines; integration with productivity APIs; admin views with charts of team activity (project analytics) <sup>3</sup> .

## 4. Personal Finance & Expense Tracker with OCR

*Project:* A budgeting and expense tracker that uses OCR to digitize receipts. Users upload or photograph receipts; the app extracts transaction data and categorizes expenses. *Components:* React or Next.js frontend for expense entry and dashboards; Node/Express backend; MongoDB for transactions; user auth. *Features:* Receipt OCR (using Tesseract or a Vision API) to parse vendor, date, and amounts (as suggested for future enhancement <sup>4</sup> ). Users can also manually add expenses, set budgets, and view charts of spending by category. An analytics dashboard (e.g. with Chart.js) displays monthly spending trends, budget vs actual, and financial insights <sup>5</sup> . Additional integrations might include bank API (Plaid) for automatic transaction import. This app highlights advanced skills (OCR, charting) and full-stack development in MERN.

- **Frontend:** React/Next.js forms and charting (e.g. Chart.js or Recharts)
- **Backend:** Express routes for expenses and OCR processing
- **Database:** MongoDB stores receipts, categories, budgets
- **Auth:** Secure user login (JWT/OAuth)
- **Key features:** Receipt upload and OCR scanning into digital transactions <sup>4</sup> ; expense categorization; budget tracking; dynamic financial charts/analytics (e.g. spending patterns) <sup>5</sup> <sup>6</sup> ; optional bank API integration.

## 5. Appointment Booking & Scheduling System

*Project:* An online scheduler for small businesses (salons, clinics, consultants). Clients can book appointments via a calendar interface. *Components:* React/Next.js frontend, Node/Express backend, MongoDB, JWT auth. *Features:* Interactive calendar or date-picker to show available slots; user registration and roles (clients vs service providers); admin interface for managing availability and appointments. Integration with Google Calendar or Office 365 API allows syncing appointments. Email/SMS reminders (via Twilio or SendGrid) can be automated on booking. Real-time slot availability prevents double-booking. This MERN app uses calendar libraries and scheduling logic, showcasing booking systems described in MERN tutorials <sup>7</sup> . It would also include an analytics dashboard of bookings over time for the business.

- **Frontend:** React/Next.js booking interface with calendar view
- **Backend:** Express APIs for appointments and availability
- **Database:** MongoDB for schedules, users, and bookings
- **Auth:** JWT/OAuth for customers and business staff

- **Key features:** Book/reschedule/cancel appointments; admin tools to block slots; calendar API integration; automated reminders; reporting on appointment stats <sup>7</sup> .

## 6. Campus Study Group Planner

*Project:* A platform for students to form and schedule study sessions or group projects. *Components:* React/Next.js frontend, Express backend, MongoDB, authentication. *Features:* Students create/join “study groups” for courses, schedule meetup times, and share resources (PDFs, links). Geolocation can help find nearby study locations (library rooms, cafes). A forum/chat for each group facilitates Q&A. Integrations could include Google Maps API (for location) and Google Calendar API (for study session invites). The app might also track participation (analytics on group meetings or study hours) and support notifications for upcoming sessions. It solves a practical campus coordination problem and demonstrates full-stack skills.

- **Frontend:** React/Next.js for group dashboards and chat
- **Backend:** Express routes for group, session, and chat data
- **Database:** MongoDB stores courses, groups, sessions, chat logs
- **Auth:** University SSO or standard JWT login for student accounts
- **Key features:** Create/join study groups; schedule study sessions; share notes/resources; integrated chat or forum for each group; geolocation/map to meeting place; notifications for new sessions.

## 7. Real-Time Chat and Collaboration App

*Project:* A full-stack chat platform (text or video) for team communication. *Components:* React/Next.js UI, Node/Express server, MongoDB, authentication, and Socket.IO or WebRTC. *Features:* User accounts and profiles, one-to-one and group chat rooms, message history stored in MongoDB. Real-time messaging via Socket.IO (as in MERN chat tutorials <sup>8</sup> ) or optional video conferencing (WebRTC). The app can support file sharing (images, docs) and typing indicators. It would require secure authentication and possibly media server setup for video. Such a project highlights real-time websockets in MERN and is a strong portfolio piece.

- **Frontend:** React/Next.js chat interface (chat windows, user lists)
- **Backend:** Node/Express with Socket.IO for real-time message delivery
- **Database:** MongoDB for user data and message logs
- **Auth:** JWT login (with email verification)
- **Key features:** Instant messaging (real-time via Socket.IO) <sup>8</sup> ; group chat rooms; file upload in chat; online/offline status; push notifications for new messages; optional video chat using WebRTC.

## 8. Local Services Marketplace

*Project:* A community marketplace connecting local service providers (plumbers, tutors, freelancers) with customers. *Components:* Next.js frontend (for SEO on listings), Express backend, MongoDB, user accounts. *Features:* Providers create listings/profiles of services (with photos, rates, service areas); customers can browse/search by category and location (using Google Maps API), read reviews, and send inquiries. Secure messaging and booking requests are built in. Integration with a payment API

(Stripe/PayPal) handles deposits or payments. This mirrors a “Freelancer Marketplace” project <sup>9</sup> but targeted to local community services. An admin dashboard tracks orders/sales and usage analytics.

- **Frontend:** Next.js for SEO-friendly listings, with responsive UI
- **Backend:** Express APIs for listings, messaging, and transactions
- **Database:** MongoDB for users, services, transactions
- **Auth:** JWT/OAuth for provider and customer accounts
- **Key features:** Service listing CRUD; search/filters (by location, price); geolocation map search; messaging between users; payment processing; admin panels; real-time notifications <sup>9</sup> .

## 9. Pet Adoption Platform

*Project:* A pet adoption portal for animal shelters and adopters. *Components:* React/Next.js frontend, Express backend, MongoDB, authentication. *Features:* Shelters or volunteers post profiles of adoptable pets (photos, descriptions, status). Users can search pets by type, age, or location. Interested adopters submit applications or inquiries through the site. An admin interface allows managing listings and adoption approvals. Integration with social media APIs (Facebook, Instagram) can help share pet profiles. Analytics can show adoption rates and most-viewed pets. This project follows the Pet Adoption idea <sup>10</sup> and demonstrates end-to-end CRUD with MERN.

- **Frontend:** React/Next.js gallery of pets and search filters
- **Backend:** Express APIs for pet listings and applications
- **Database:** MongoDB for pets, shelters, adopters
- **Auth:** JWT sign-up for shelter staff and users
- **Key features:** Pet listing management (add/edit/remove); search and filters; application/inquiry forms; user dashboards; image upload; notifications for new applications; admin adoption tracking <sup>10</sup> .

## 10. Small Business Inventory & Sales Dashboard

*Project:* A point-of-sale and inventory tracker for a small retail business (cafe, store). *Components:* React/Next.js frontend, Node/Express backend, MongoDB, authentication. *Features:* Product management (add items, stock levels, pricing), sales entry, and inventory updates. The app could include barcode scanning (mobile camera or webcam) for quick input. Integration with a payment terminal or API (Square/Stripe) records transactions. A real-time analytics dashboard charts sales by day, best-selling items, and inventory alerts for low stock (using Chart.js or similar <sup>5</sup> ). Optional features: CRM for customers (loyalty tracking) or integration with accounting software. This full-stack project emphasizes data visualization and automation, rounding out a portfolio with a business application.

- **Frontend:** React/Next.js with inventory and POS interface
- **Backend:** Express APIs for products, sales, and reports
- **Database:** MongoDB for inventory, sales records, user accounts
- **Auth:** Secure admin login (JWT)
- **Key features:** Product catalog and stock management; sales entry (with barcode scan); automated stock decrement; dashboard of charts (daily revenue, stock alerts) <sup>5</sup> ; optional API integration for payments and accounting.

Each idea above is a full MERN (or Next.js/Node) stack application with authentication, and most include advanced features (maps, OCR, realtime, analytics) that make them portfolio-worthy. They tackle real problems for communities, businesses, or students, demonstrating practical, deployable web solutions.

**Sources:** Concepts and inspiration are drawn from MERN project tutorials and idea lists <sup>1</sup> <sup>2</sup> <sup>5</sup> <sup>8</sup> <sup>7</sup> <sup>9</sup> , ensuring these ideas leverage current technologies and patterns.

---

<sup>1</sup> Volunteer Management System using MERN Stack - GeeksforGeeks  
<https://www.geeksforgeeks.org/mern/volunteer-management-system-using-mern-stack/>

<sup>2</sup> Neighborhood Safety Rating App with MERN Stack - GeeksforGeeks  
<https://www.geeksforgeeks.org/mern/neighborhood-safety-rating-app-with-mern-stack/>

<sup>3</sup> <sup>6</sup> <sup>7</sup> <sup>8</sup> <sup>9</sup> <sup>10</sup> 40 MERN Stack Projects With (Source Code) In 2025  
<https://www.fynd.academy/blog/mern-stack-projects>

<sup>4</sup> <sup>5</sup> Mern Stack Finance Tracker with Real Time-Analytics  
<https://www.ijssred.com/volume8/issue3/IJSRED-V8I3P423.pdf>